PREPSENSE

Team Work Plan & Delivery Roadmap

A practical, five-person plan for building, testing and improving the company-aware AI interview preparation system.

PROJECT GOAL

Turn a job description, company research and a candidate resume into a focused interview session - then evaluate answers with transparent, evidence-based feedback.

How to use this document

- Every member should read the team ownership page and the week-by-week plan.
- Use your named workstream as the primary ownership area; coordinate before changing shared interfaces.
- A task is complete only when its code, tests, documentation and integration evidence are available.
- Weeks 1–2 describe the current foundation and stabilization work. Weeks 3–5 describe the planned next phases.

Team structure

Five members, five clear workstreams: orchestration, research, question generation, answer evaluation, and frontend plus evaluation operations.

Prepared: 4 November 2026 | Project repository: github.com/kashardagdi/PrepSense

SHARED UNDERSTANDING

1. Project snapshot

PrepSense is an agentic interview-preparation application. It uses a candidate resume, a target company and a job description to prepare relevant interview questions. Candidates submit answers and receive structured feedback and scores.

Current product workflow

Step	Stage	What happens
1	Inputs	Company name + job description + resume PDF
2	Research	Tavily search retrieves company information and source links
3	Understanding	Resume text extraction and job-description parsing
4	Preparation	Qwen3:8B generates 8 questions: 5 technical + 3 behavioral
5	Practice	Candidate selects a generated question and submits an answer
6	Evaluation	Version 3 evaluator scores rubric criteria and returns evidence and feedback

Technology baseline

Area	Current technology / implementation
Application	Python + Streamlit
Local model	Ollama with Qwen3:8B; shared ask_ai() wrapper in llm.py
Research	Tavily search
LLM framework	LangChain / ChatOllama
Resume processing	pypdf
Evaluation	Python rubric validation + Qwen-generated criterion scores
Tests	pytest foundation to be completed and expanded
Source control	GitHub; protected main branch with PR workflow

Current baseline: what has been built

- Resume PDF text extraction, job-description parser, company research and summarization.
- Resume-aware question generator with an 8-question target (5 technical, 3 behavioral) and JSON validation.
- Answer evaluator Version 3 with separate technical and behavioral rubrics, score breakdown and evidence.
- Integrated pipeline and Streamlit UI for interview generation, question display, answer submission and evaluation feedback.

Important baseline observation

A terminal run measured approximately 312 seconds through question generation: company summary 124.63s, JD parsing 58.23s and question generation 125.99s. This is a measured baseline from one run, not a guaranteed runtime. Performance optimization is an explicit task.

FIVE WORKSTREAMS

2. Team ownership

Each member owns a workstream across all weeks. Ownership means implementation, tests, documentation and bringing the work to integration-ready status - not working in isolation.

MEMBER 1	**LangGraph Architect & Integration Lead** Owns shared architecture, state schema, graph orchestration, interfaces, integration, end-to-end tests and release coordination. Branch: feat/langgraph-orchestration
MEMBER 2	**Research Agent & Source Quality** Owns Tavily research, company identity/source relevance, job context extraction, evidence quality and research-agent tests. Branch: feat/researcher-agent
MEMBER 3	**Question Generation Agent** Owns resume-aware question prompts, question quality, 5+3 distribution, duplicate/unsupported-question checks and generator tests. Branch: feat/generator-agent
MEMBER 4	**Answer Evaluation Agent** Owns technical/behavioral rubrics, evidence-based scoring, STAR feedback, calibration datasets and evaluator tests. Branch: feat/evaluator-agent
MEMBER 5	**Frontend & Evaluation Operations** Owns Streamlit interview experience, answer submission, progress and feedback UI, evaluation runner/dashboard and experiment reporting. Branch: feat/frontend-evaluation

Shared responsibility

Every member must write or maintain tests for their own module, document its interface, review at least one teammate PR per week where possible, and help resolve integration defects in their area.

CURRENT PHASE

3. Weeks 1–2: foundation and stabilization

Status

The core application flow and Streamlit answer-evaluation integration have been implemented. The work below is the ownership-based stabilization checklist; confirm actual individual contribution with the team rather than assuming all tasks were completed by the same person.

Owner	Work to own / stabilize	Evidence to produce
Member 1 - Integration	Maintain llm.py shared ask_ai() interface and pipeline.py; document function contracts; keep terminal and Streamlit paths consistent; capture end-to-end timing baseline.	Working pipeline; interface notes; baseline timing log; smoke test.
Member 2 - Research	Maintain Tavily company search and company summarization; review company identity and source relevance; review JD parsing fields and resume extraction handoff.	Research module; source-quality checklist; JD extraction sample tests; known-issue list.
Member 3 - Generator	Maintain 8-question generation; ensure 5 technical + 3 behavioral; apply resume-grounding rules; validate schema, duplicates and question usefulness.	Generator module; prompt/rules documentation; sample outputs; validation tests.
Member 4 - Evaluator	Maintain Version 3 technical/behavioral rubric; ensure final score is computed in Python; reject invalid rubric output; run weak/average/strong calibration.	Evaluator module; rubric definition; calibration results; invalid-output tests.
Member 5 - Frontend	Maintain Streamlit layout; connect generation and answer submission; display score breakdown and feedback; verify session state and user-facing errors.	Working Streamlit flow; UI smoke test; screenshots or demo notes; issue list.

Week 1–2 exit criteria

- One user can submit a company, JD and resume and receive exactly 8 questions (5 technical, 3 behavioral).
- Each question can be answered in Streamlit and evaluated through the Version 3 evaluator.
- Technical and behavioral evaluation outputs contain score, breakdown, evidence, strengths, missing points and suggestions.
- Invalid inputs, unreadable PDFs and malformed evaluator JSON produce clear errors - never a fabricated default score.
- All long-running stages have timing logs; the team records the same input set for repeatable comparisons.
- Company research findings are tied to relevant sources; unrelated company claims are flagged rather than passed to question generation.

Known issues to carry forward

The observed Microsoft research example included unrelated Vexcel material. The evaluator’s technical calibration improved to 3/10 weak, 7/10 average and 8/10 strong on one test question; behavioral calibration and broader consistency testing remain open.

BUILD THE MULTI-AGENT WORKFLOW

4. Week 3: agent architecture and orchestration

Goal: move the stable backend functions into clear agent modules and orchestrate them with LangGraph. Do not rewrite working logic just to move files; first wrap existing functions behind agreed interfaces, then migrate incrementally.

Target workflow

Stage	Responsibility
Input / State	company_name, job_description, resume_text
Research + JD understanding	Researcher agent gathers company evidence; job parser extracts role requirements
Question generation	Generator agent receives validated company, job and resume context
Interview session	Streamlit displays 8 questions and collects answers
Answer evaluation	Evaluator agent scores one submitted answer using the agreed rubric

Member-by-member tasks

Owner	Tasks	Week 3 deliverables
Member 1	Define shared TypedDict state; create graph nodes/edges; add conditional path for answer evaluation; integrate all agents; handle state updates, errors and graph tests.	Runnable graph; state schema; graph diagram; integration test.
Member 2	Create agents/researcher.py; wrap company search and job understanding; preserve source URLs; improve entity matching and irrelevant-source rejection.	Researcher node; source metadata; research tests with mocked Tavily.
Member 3	Create agents/generator.py; wrap current question generator; enforce 8 questions and 5+3 mix; validate fields, grounding, duplicates and question clarity.	Generator node; schema validation; generator tests.
Member 4	Create agents/evaluator.py; wrap Version 3; preserve rubric breakdown/evidence; support technical and behavioral questions; validate score totals.	Evaluator node; rubric tests; invalid-response tests.
Member 5	Connect Streamlit to graph entry point; show progress by stage; keep answer forms tied to question IDs; display evaluation feedback and prevent stale results.	Graph-backed UI; stage status; answer/evaluation demo.

Week 3 acceptance criteria

- Research, generator and evaluator components execute through the graph - not only as separate scripts.
- State contains only the agreed shared fields; nodes do not depend on undocumented globals.
- Graph returns the expected 8 questions and preserves the 5 technical / 3 behavioral split.
- A selected question and answer reach the evaluator with the correct question type and job context.
- Streamlit shows which stage is running and displays structured evaluation feedback.
- Unit tests use mocked LLM and Tavily calls; one central integration run uses the real local model and Tavily.

PROVE THAT THE AGENTS ARE USEFUL

5. Week 4: quality evaluation and reliability

Goal: stop judging the system only by whether it runs. Build a repeatable evaluation set and measure research accuracy, question quality and answer-scoring consistency.

Shared evaluation dataset

Dataset item	Plan
Job postings	20–30 representative job descriptions across relevant roles/experience levels.
Contributions	Each member contributes 4–6 examples or review cases.
Question review	Rate relevance, specificity, resume alignment, factual grounding and usefulness on a 1–5 scale.
Answer review	For selected technical and behavioral questions, prepare weak / average / strong sample answers and expected rubric reasoning.
Manual review	Manually inspect at least 5–10 generated sessions and record concrete defects.
Storage	Keep test inputs, expected criteria and result logs in evaluation/ using JSON or CSV plus a short methodology note.

Member-by-member tasks

Owner	Tasks	Week 4 deliverables
Member 1	Build an evaluation runner that calls the integrated graph on fixed cases; capture timings, outputs, failures and version/config metadata; coordinate the baseline comparison.	Repeatable evaluation script; consolidated report.
Member 2	Audit company research identity, source relevance, source recency and factual support; track false matches and unsupported claims; improve search strategy.	Research quality results; source audit examples; fixes.
Member 3	Review generated questions for role fit, specificity, resume grounding, duplication, difficulty and unsupported experience assumptions.	Question quality scores; prompt/validator improvements.
Member 4	Calibrate rubric scoring across several question types; check evidence accuracy; compare model marks with human-reviewed marks; test repeated-run consistency.	Evaluator calibration report; revised rubric/tests.
Member 5	Build or maintain a dashboard/report view for evaluation results; collect reviewers' 1–5 ratings; summarize trends and failures.	Evaluation dashboard or report; review log.

Metrics to report

- Researcher: correct company identity, relevant sources, source-backed claims and unrelated-source rate.
- Generator: role relevance, resume alignment, duplicate rate, unsupported assumptions and valid 5+3 question mix.
- Evaluator: rubric validity, evidence accuracy, weak < average < strong ordering and agreement with human review.
- System: per-stage and end-to-end latency, failures, invalid JSON and duplicate calls.

Week 4 exit criteria

The team can rerun the same evaluation set after a change and compare results. Every quality claim in the demo is supported by test cases or manual review, not by a single impressive example.

EXPERIMENT ONLY AFTER THE BASELINE IS STABLE

6. Week 5: optional fine-tuning experiment

Scope guardrail

LoRA / QLoRA is an optional research experiment. It must not delay the working PrepSense application, agent integration or evaluation report. If the dataset or training environment is not ready, document the experiment plan and keep the prompted Qwen baseline.

Goal: investigate whether a small adapter fine-tune improves answer-feedback consistency or another measurable evaluator quality metric compared with the existing prompted Qwen3:8B baseline.

Training data plan

- Target 100–200 curated examples, if the team can prepare them with sufficient quality.
- Example structure: question + answer + job/company context + expected feedback / rubric labels.
- Include technical and behavioral samples, score breakdowns, evidence, missing points and improvement suggestions.
- Use a held-out test split. Do not evaluate on examples used to train the adapter.
- Use synthetic, public, or consented and redacted examples. Do not place private resumes or personal candidate data into shared training files without permission.

Member-by-member tasks

Owner	Tasks	Week 5 deliverables
Member 1	Define the experiment interface; keep baseline inference available; integrate an adapter only behind a switch; compare baseline and adapter on identical held-out cases.	A/B comparison; safe fallback to baseline.
Member 2	Prepare company/JD-grounded context examples; verify source-grounded facts and role context; remove unrelated or unsupported claims.	Curated context examples with provenance.
Member 3	Prepare question, answer and context examples that cover different skill levels and avoid repetitive templates.	Curated technical / behavioral examples.
Member 4	Create expected rubric scores, criterion evidence, STAR labels and feedback targets; review label consistency across examples.	Quality-controlled feedback labels.
Member 5	Build the Colab training and experiment workflow using a suitable Hugging Face PEFT / TRL approach; record config, logs and adapter artifact.	Reproducible training notebook and experiment record.

How to decide whether fine-tuning helped

Compare	Requirement
Baseline vs adapter	Use the same questions, answers, context and held-out cases.
Quality	Compare rubric correctness, evidence quality, score ordering and human-review agreement.
Reliability	Check JSON validity, missing fields and inconsistent feedback.
Performance	Record latency and token counts, but do not trade away evaluation quality for speed without reporting the trade-off.
Decision	Keep the adapter only if it produces a clear, repeatable improvement. Otherwise retain the prompted baseline.

PERSONAL ROADMAP

7. Each member: responsibility across all weeks

Use this page to understand how your workstream evolves. The same owner remains accountable from the first implementation through integration and evaluation.

Owner	Weeks 1–2	Week 3	Week 4	Week 5
Member 1 Integration Lead	W1–2: own shared interfaces, llm.py, pipeline and integration baseline.	W3: build LangGraph state, nodes, edges, error paths and graph tests.	W4: run repeatable end-to-end evaluation; consolidate timing and quality results.	W5: compare baseline vs adapter; integrate only if measurable improvement.
Member 2 Research	W1–2: maintain Tavily research, JD/resume context handoff and source checks.	W3: deliver researcher agent with grounded source metadata and relevance checks.	W4: audit company identity, factual support, source quality and false matches.	W5: curate company/JD-grounded training examples if fine-tuning proceeds.
Member 3 Generator	W1–2: maintain resume-aware 8-question generator and structured output validation.	W3: deliver generator agent with 5+3 balance, grounding and duplicate checks.	W4: measure question relevance, specificity, difficulty and personalization.	W5: curate diverse question/answer/context examples if fine-tuning proceeds.
Member 4 Evaluator	W1–2: maintain Version 3 rubric, score calculation, evidence and technical calibration.	W3: deliver evaluator agent for technical and behavioral answers.	W4: test score consistency, evidence accuracy and human-review agreement.	W5: prepare expected scores, criterion evidence and STAR labels if fine-tuning proceeds.
Member 5 Frontend + Eval Ops	W1–2: maintain Streamlit generation, answer submission and feedback display.	W3: connect UI to graph; show stage progress and per-question evaluation.	W4: collect reviews and display evaluation results in a dashboard/report.	W5: prepare Colab training workflow and record experiment outputs.

Handoffs and dependencies

- Member 2 provides validated company and job context before Member 3 finalizes the question-generation input contract.
- Member 3 provides stable question objects including question text and type so Member 5 can render answer forms and Member 4 can evaluate the correct type.
- Member 4 publishes the evaluator return schema before Member 5 finalizes the feedback UI.
- Member 1 approves shared state and interface changes before agents are connected into the graph.
- Member 5 reports usability and UI integration defects; the relevant backend owner fixes the underlying module rather than hiding the defect in the UI.

Rule for shared interfaces

Agree on function signatures, required keys and return types before parallel coding. If an interface must change, announce it, update the contract, tests and dependent modules in the same PR or coordinated PR set.

HOW THE TEAM WORKS

8. GitHub, testing and collaboration rules

Branch and pull-request workflow

1. Pull the latest main before starting a task.
2. Create or switch to your assigned feature branch.
3. Work only in your owned files unless a shared change is agreed.
4. Run your unit tests and relevant formatting / syntax checks locally.
5. Commit small, descriptive changes and push your branch.
6. Open a pull request to main. Explain what changed, how it was tested and any known limitation.
7. Ask another member to review. Resolve comments and confirm CI/tests pass.
8. Merge only through the repository ruleset; do not bypass protected main.

How everyone tests without installing Qwen3:8B

Qwen3:8B is available on the integration laptop. Other members can still test their code using pytest and mocks. Unit tests should replace ask_ai() and Tavily calls with deterministic fake responses. The integration lead runs the real Ollama + Tavily end-to-end checks.

Test level	Who / where	What it proves
Unit tests	Every member; pytest with mocked LLM/search	Input validation, prompt construction, parsing, schema and rubric logic.
Component tests	Module owner; mocked external dependencies	The module returns the agreed interface and handles failure cases.
Graph tests	Member 1 plus owners; mocked nodes where possible	State moves through nodes and conditional branches correctly.
Real integration	Member 1 on local Ollama + Tavily machine	Actual model responses, end-to-end flow, latency and external search behaviour.
UI smoke test	Member 5 plus one reviewer	Form submission, progress display, question selection, answer submission and feedback.

Proposed repository structure

```
PrepSense/
  agents/
    state.py
    researcher.py
    generator.py
    evaluator.py
  tests/
    test_researcher.py
    test_generator.py
    test_evaluator.py
    test_graph.py
    test_pipeline.py
  evaluation/
    test_dataset.json
    scoring_rubric.md
    evaluation_report.md
  llm.py
  pipeline.py
  app.py
  requirements.txt
```

Create files when the corresponding work begins; do not add empty placeholder modules just to match the tree.

RELEASE DISCIPLINE

9. Definition of done and team cadence

A task is complete only when...

- Implementation is committed on the owner's feature branch.
- Unit tests cover normal inputs, invalid inputs and important edge cases.
- LLM and Tavily dependencies are mocked in unit tests; no API key or local model is required for basic teammate testing.
- Return data matches the agreed interface and is documented with an example.
- Logs and error messages are useful but do not expose secrets, full private resumes or API keys.
- A PR has a short description, test evidence and reviewer approval.
- The integrated application has been smoke-tested after merge.

Suggested team rhythm

Cadence	Activity	Expected output
Start of week	30-minute planning: confirm owners, acceptance criteria and dependencies.	Tasks sized and branches agreed.
During week	Short async progress update: done / next / blocked. Raise interface conflicts early.	Visible progress; blockers assigned.
Mid-week	Integration checkpoint for interfaces and sample outputs.	No late surprises at merge time.
End of week	Demo, test review, merge and retrospective.	Working increment and evidence against acceptance criteria.

Security and data handling

- Never commit .env files, Tavily API keys, passwords or tokens. Keep .env in .gitignore.
- Do not add real resumes to shared repositories or fine-tuning datasets unless the candidate has explicitly consented and personal data is appropriately redacted.
- Use synthetic examples for demos and tests wherever possible.
- Treat retrieved webpages and resume text as untrusted data, not as instructions to the agents.

Team sign-off

At the end of each week, the team should be able to answer: What works now? What tests prove it? What is still unreliable? What is the next owner handoff?

Final project principle

Build a reliable interview-preparation product first. Add more agents or fine-tuning only when they improve measurable quality, reliability or usability.

Document owner: PrepSense project team | Roadmap version: 1.0 | Prepared 4 November 2026